

PCS956-TS-companion_C

September 24, 2026

0.1 PCS956 time series companion C: anomaly detection, concept drift, multivariate dependence, ML with context, and explainability

This companion notebook provides **code templates** for topics discussed in lecture TS3:

- anomaly detection based on residuals and thresholds;
- concept drift monitoring and simple retraining strategies;
- multivariate dependence and cross-correlation;
- supervised learning on lagged plus exogenous/context features;
- simple feature-importance based explainability for tree-based models;
- scaffolding for simulation-based mini-projects, including synthetic data.

Each **code cell is self-contained**: it includes its own imports and helper functions, so you can copy it into a fresh notebook without needing other cells. Illustrative examples and extended discussion live in the TS3 lecture notebook.

You are expected to:

- adapt these templates to your own dataset or simulations;
- define clear train/validation/test splits, with gaps where appropriate;
- compare anomaly and drift methods against simple baselines (from Companions A and B);
- be explicit about the limitations of each method and the data.

The examples in this companion use simulated data. If you want to use one of the templates for your mini-project or other applied work, you must replace the simulation step with code that reads your time series (see Companion A).

0.1.1 1. Residual-based anomaly flags for a univariate series

In many applications, anomalies are observations that are **implausible under some notion of ‘normal behaviour’**. There are several ways to formalise this:

- **Model-free rules**, for example:
 - values outside a physically meaningful range (for example negative counts, or heart rates above a plausible maximum),
 - jumps or local changes that are too fast to be realistic for the system (for example GPS altitude changes corresponding to jumping off a cliff when no such event is possible),
 - unusually large first differences relative to typical variability.
- **Model-based residuals**, where we:
 - fit a simple model to ‘normal’ data (for example a baseline forecasting model),
 - define anomalies as points where the residuals (observed minus predicted) are unusually large, given the estimated noise level.

The template in this section focuses on the **model-based residual** view, because it fits naturally with the forecasting workflow in Companions B and C. It should be seen as one practical approach among many, not as a universal definition of anomalies.

You can combine this with simpler checks on levels and differences: for example, you might mark any observation as anomalous if it has both a large residual *and* a physically implausible jump compared to the previous few points.

Usage pattern:

1. Choose a training window where behaviour is believed to be relatively normal.

2. Fit a simple ARIMA model:

```
model, results = fit_simple_arma(train_series, order=(1, 0, 0))
```

3. Compute residuals on a later window:

```
resid = multi_step_residuals(results, eval_series)
```

4. Flag anomalies:

```
flags = flag_anomalies(resid, k=3.0)
```

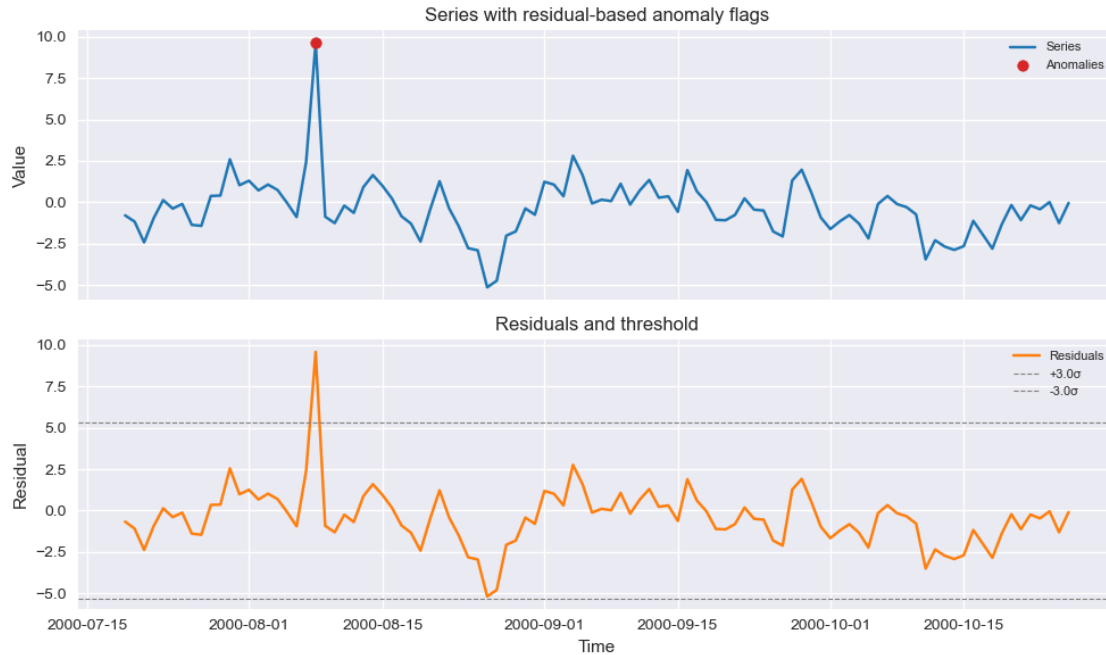
5. Visualise:

```
plot_anomalies(eval_series, resid, flags, k=3.0)
```

Threshold choices are subjective and should be documented and justified in your mini-project. Remember that the residual standard deviation is estimated from the residuals you pass in (for example from a calibration window); if the residual scale changes over time due to drift, fixed thresholds may become unreliable.

Example: residual-based anomalies on a synthetic series with a spike

The following self-contained example simulates an AR(1) series, injects a single large spike as an anomaly, and applies the residual-based anomaly detector from a ‘normal’ training window to a later evaluation window.



0.1.2 2. Concept drift monitoring via block-wise performance

Concept drift refers to changes in the underlying behaviour of the process we are modelling. This can include:

- shifts in the distribution of the target (for example changes in mean, variance, or seasonality);
- changes in how predictors relate to the target (for example a feature that used to be informative stops being useful);
- broader changes in the data-generating mechanism (for example new policies, technologies, or user behaviours).

When such changes occur, a model trained on past data can become misaligned with current behaviour, and its predictive performance may deteriorate. Monitoring model performance over time is therefore a practical way to track drift: persistent drops or trends in error metrics can be treated as signals that the underlying behaviour has changed enough to warrant re-inspection or retraining.

The template below implements a simple block-wise performance monitor using an expanding training window and fixed-size evaluation blocks.

Usage pattern:

- Decide on a block size (for example 30 days, 12 months).
- Evaluate performance over time:

```
results_df = block_forecast_evaluation(
    series,
    block_size=30,
```

```

model_fn=arima_model_fn,
model_kwargs={'order': (1, 0, 0)},
)

```

- Plot performance:

```

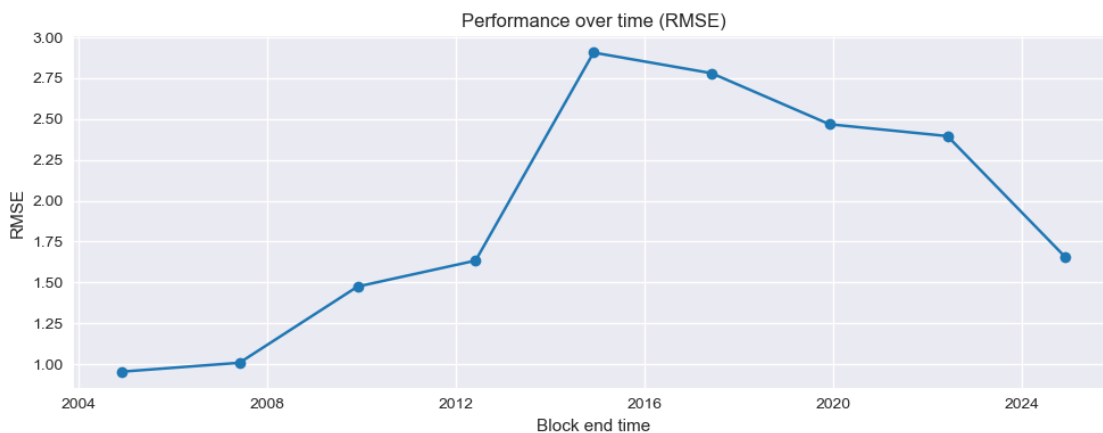
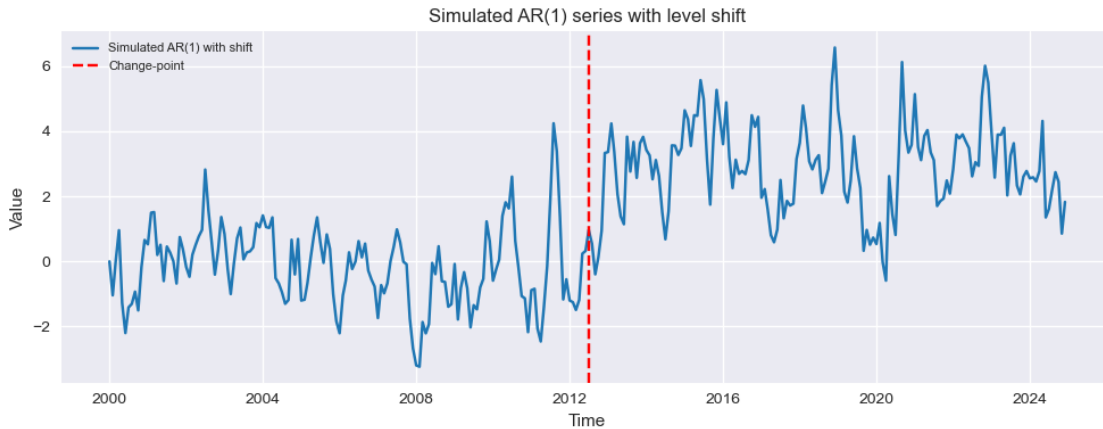
plot_block_performance(results_df, metric='RMSE')

```

Drops or trends in performance can be treated as potential drift signals. In simulation-based mini-projects, you can generate series with known drift and check that performance curves respond as expected.

Example: concept drift in an AR(1) series with a level shift

This example simulates an AR(1) series with a level shift halfway through, then uses block-wise expanding-window evaluation to show how RMSE changes over time. We first plot the series itself (with the change-point marked), then the RMSE by block.



0.1.3 2.1 Optional: using `auto_arima` to select ARIMA order

In the examples above we used an $\text{ARIMA}(1, 0, 0)$ model. This was a deliberate simplification: it keeps the focus on residuals, drift, and evaluation, without adding the extra complexity of model selection.

In realistic applications we do not know the true ARIMA order. A common strategy is to let a library search over a small set of candidate orders and choose the one that optimises an information criterion such as AIC or BIC. This is still not automatic truth, but it provides a transparent starting point.

The `pmdarima` package provides an `auto_arima` function that:

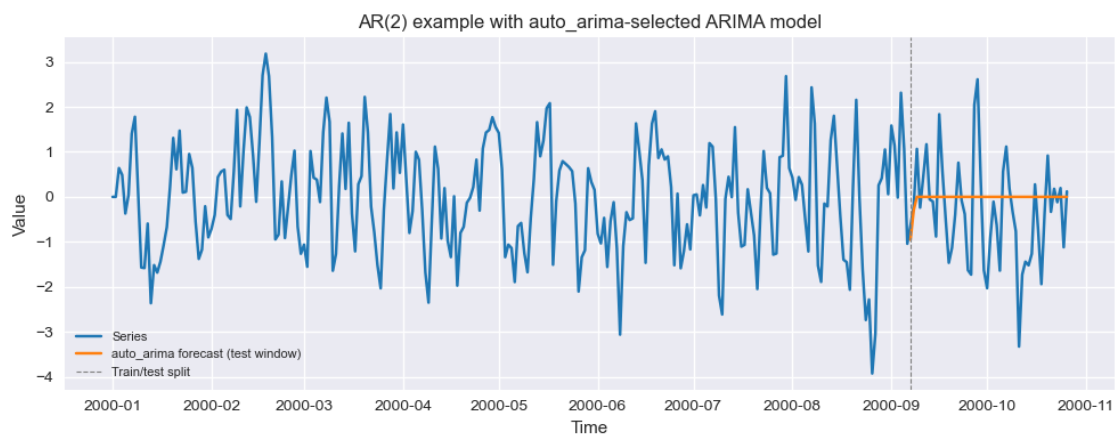
- tests differencing orders d (and seasonal differencing if requested);
- explores a grid of (p, d, q) (and optionally seasonal (P, D, Q, m));
- selects a model using an information criterion (AIC by default).

The template below shows a minimal usage pattern on a synthetic AR(2) series. You can adapt the ranges and options for your own data. This is intended as a convenience tool, not a replacement for domain knowledge or careful diagnostics.

Note: you will need to install `pmdarima` (for example with `pip install pmdarima`) to run this section.

Selected order (p, d, q): (0, 0, 2)

AIC of selected model: 867.7377050073519



0.1.4 3. Multivariate dependence: cross-correlation and Granger-style tests

For multivariate time series it is often useful to understand how variables move together over time and whether one series helps to predict another.

- **Cross-correlation functions (CCFs)** measure (linear) correlation between two series at different lags:
 - a positive lag (for example lag = +3) asks: how well does the past of series X line up with the present of series Y (X leading Y)?

- a negative lag asks the opposite (Y leading X).
- this is symmetric and descriptive: it tells you about linear dependence patterns, not about causal direction.
- **Granger-style tests** ask a predictive question:
 - does including past values of series X improve the prediction of series Y, beyond using past values of Y alone?
 - if so, we say ‘X Granger-causes Y’ in a predictive sense.
 - this is a stronger statement than simple correlation, but still about prediction, not about true causal mechanisms in the physical or social sense.

The templates below provide:

- a helper to compute and plot cross-correlation between two series;
- a small wrapper to run Granger-style predictive tests using statsmodels.

They are intended as **exploratory tools**. Interpretation should be cautious, and ideally supported by domain knowledge and additional modelling.

Note that cross-correlations at large lags are based on fewer overlapping observations and can be unstable. It is often sensible to restrict attention to moderate lags where a reasonable amount of data is available.

Usage pattern:

- Align two series on a common index and consider appropriate transforms (for example differences or residuals):

```
plot_cross_correlation(series_x, series_y, max_lag=40, label_x='X', label_y='Y')
```

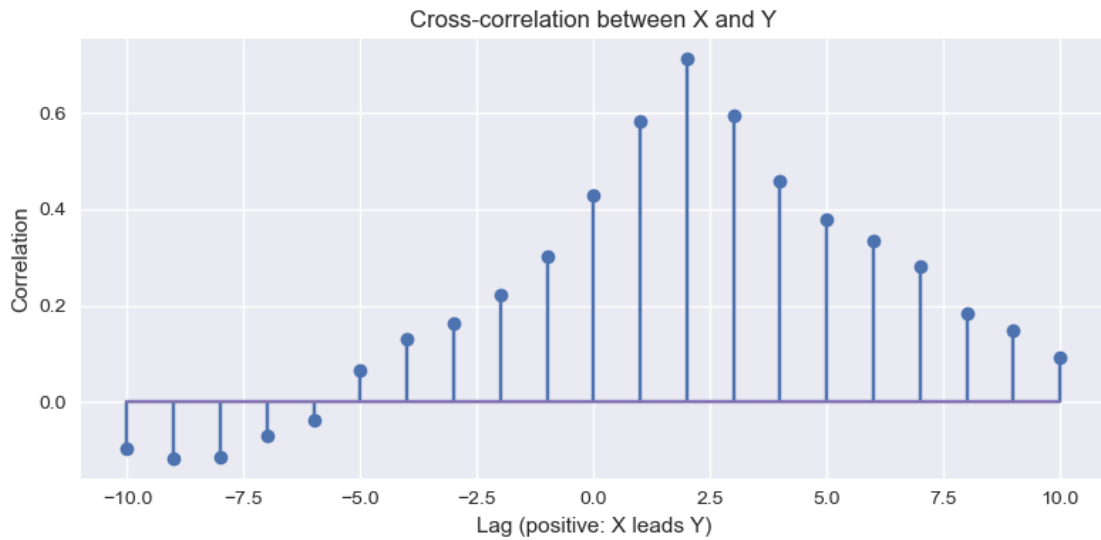
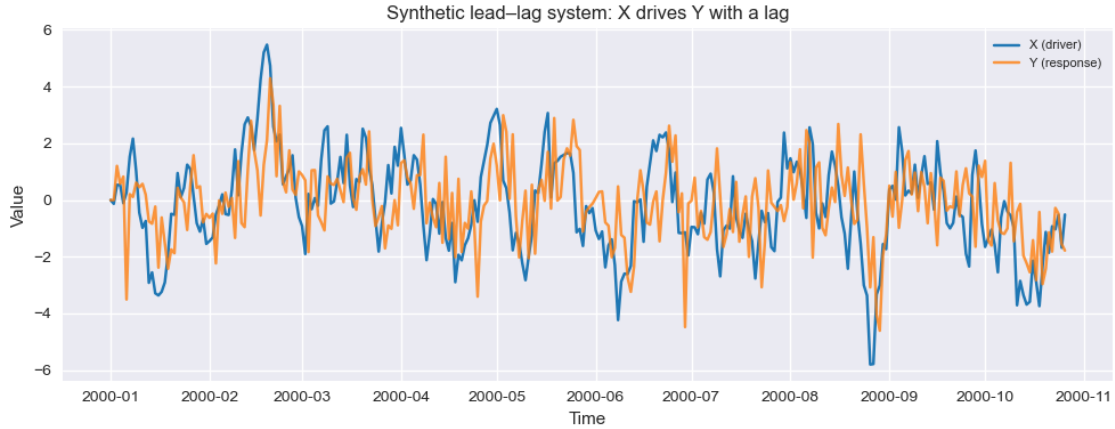
- For Granger-style tests:

```
df_xy = pd.DataFrame({'Y': series_y, 'X': series_x})
results = granger_test(df_xy, cause_col='X', effect_col='Y', maxlag=4)
```

Granger-style predictive influence is about forecasting improvement, not definitive causal effect. Use these tools as initial probes, not final answers.

Example: cross-correlation for a synthetic lead-lag relationship

The following example constructs two synthetic series where one is designed to respond (with lag) to the other. We first plot the two series to see the lead-lag structure, and then plot the cross-correlation function, which highlights the same pattern.



Example: Granger-style test on a synthetic bivariate system

We reuse the synthetic series X and Y from the previous example, where X was constructed to drive Y with a lag. We now apply a simple Granger-style predictive test to see whether including past X improves prediction of Y , beyond using past Y alone.

For each lag $l = 1, 2, \dots, \text{maxlag}$, the `grangercausalitytests` function runs an F-test for the null hypothesis:

- H_0 : past values of X up to lag l do not help to predict Y , beyond using past Y alone;
- H_1 : past values of X up to lag l do help to predict Y .

We typically fix a significance level, for example $\alpha = 0.05$:

- if $p\text{-value} < \alpha$ we reject H_0 at lag l and conclude that X Granger-causes Y (in a predictive sense) at that lag;

- if $p\text{-value} \geq \alpha$ we do not reject H_0 at lag l .

This is about short-term predictive value, not about true causal mechanisms. In this synthetic setting, X is designed to influence Y , so we expect to see evidence against H_0 at some lags, but this should be interpreted as a predictive relationship, not as a full causal story about a real system.

P-values for H_0 : "X does NOT Granger-cause Y" (F-test):

```
lag 1: p = 5.124e-10 -> REJECT H0 (evidence that X Granger-causes Y)
lag 2: p = 2.929e-09 -> REJECT H0 (evidence that X Granger-causes Y)
lag 3: p = 1.411e-08 -> REJECT H0 (evidence that X Granger-causes Y)
lag 4: p = 3.942e-08 -> REJECT H0 (evidence that X Granger-causes Y)
```

0.1.5 4. Supervised learning with lagged plus exogenous features

Many forecasting and anomaly-detection problems use both lagged values and exogenous or context variables (for example calendar effects, weather, or regime indicators).

This template extends the lagged-feature construction from Companion B to include exogenous inputs.

Usage pattern:

- Construct a target series and an exogenous DataFrame with aligned indices:

```
target = my_df['y']
exog = my_df[['dow', 'holiday', 'temp']]
```

- Build supervised data:

```
X, y, feature_names = series_and_exogenous_to_supervised(target, exog, n_lags=14, horizon=1)
```

- Split (X, y) into train/validation/test according to time (for example using the same positional split pattern as in Companion B), then fit and evaluate:

```
model = rf_model_exog_fn(X_train, y_train)
y_pred = model.predict(X_test)
```

This template is intended for forecasting tasks with exogenous inputs; similar patterns can be adapted for anomaly detection and drift monitoring.

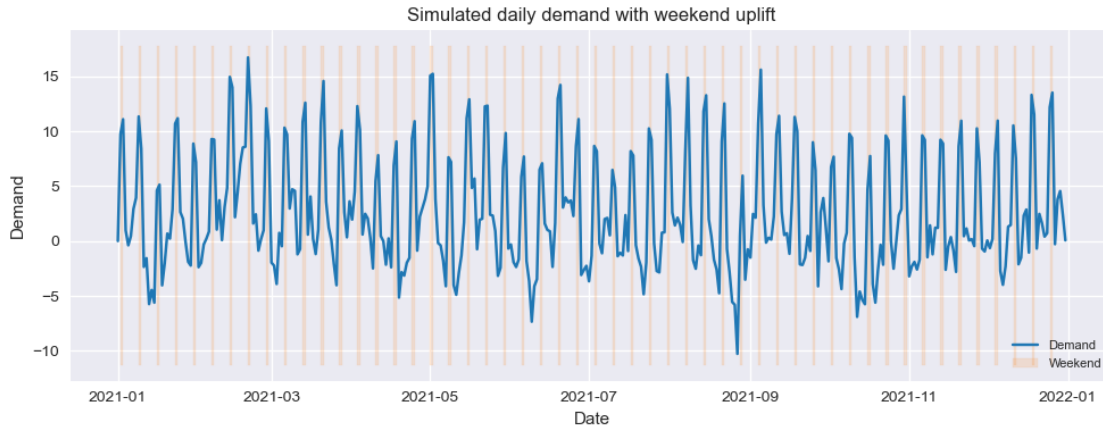
Example: daily demand forecasting with a weekend effect (RandomForest and Gradient Boosting)

This example simulates a daily demand series with an AR(1) structure and a weekend uplift. We fit two modern tree-ensemble models on lagged demand values plus a simple binary ‘is_weekend’ context feature:

- RandomForestRegressor (bagging-style trees),
- HistGradientBoostingRegressor (gradient boosting on histograms).

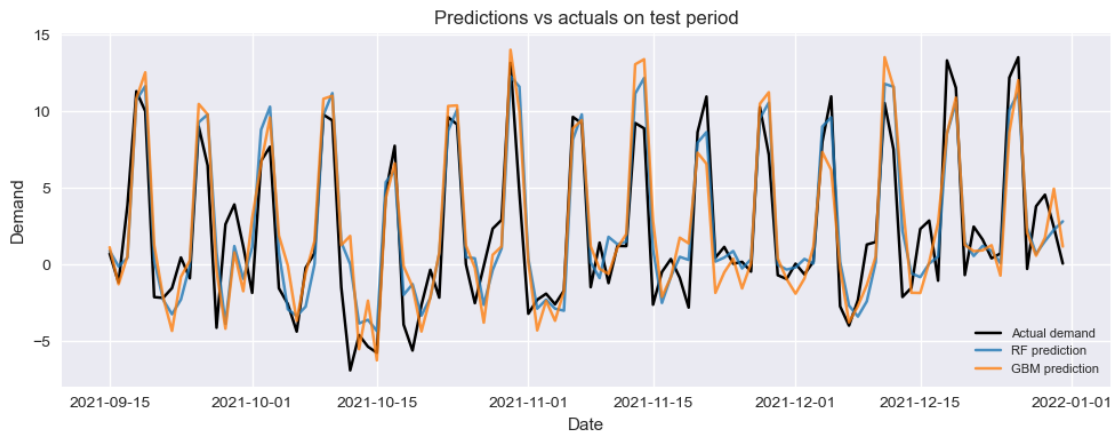
We first plot the simulated series, then compare predictions on the test period.

Depending on the random seed and hyperparameters, either the Random Forest or the gradient boosting model may perform slightly better on this toy dataset. Here the Random Forest has a lower MAE, but the main point is that both models fit naturally into the same lag-plus-exogenous pipeline.



RandomForest Test MAE: 1.8693232354025382

GradientBoost Test MAE: 2.1154332033930734



0.1.6 5. Feature importance for tree-based models on lagged and context features

Explainability for tree-based models can start from feature importance scores. This template provides:

- a helper to compute feature importances from a fitted RandomForest;
- a plotting function to visualise importances;
- an optional aggregation of importances for lagged vs exogenous features.

Note on ‘impurity-based’ importance

For tree-based models (for example RandomForest and gradient boosting), each split in a tree is chosen to make the target values in the child nodes more homogeneous than in the parent node. A measure of how mixed or heterogeneous a node is called an impurity measure (for regression, this is often the variance of the target in that node).

Impurity-based feature importance works by:

- looking at how much each split reduces impurity (for example how much the variance of the target decreases after the split), and
- attributing that reduction to the feature used in the split,
- then summing these reductions over all splits and all trees for each feature.

The built-in `feature_importances_` attribute in scikit-learn reports these aggregated impurity reductions, normalised to sum to 1 across features.

This is convenient and fast, but it has limitations. In particular, it can over-favour features that have many distinct values (many possible split points) compared to binary or low-cardinality features. For more robust interpretability it is often useful to complement these scores with permutation importance, which measures how much the model's error increases when a feature is randomly shuffled in the test data.

Usage pattern:

- After fitting a RandomForest model on lagged+exogenous features, build a feature importance table:

```
df_imp = feature_importance_df(model, feature_names)
plot_feature_importance(df_imp, top_k=20)
```

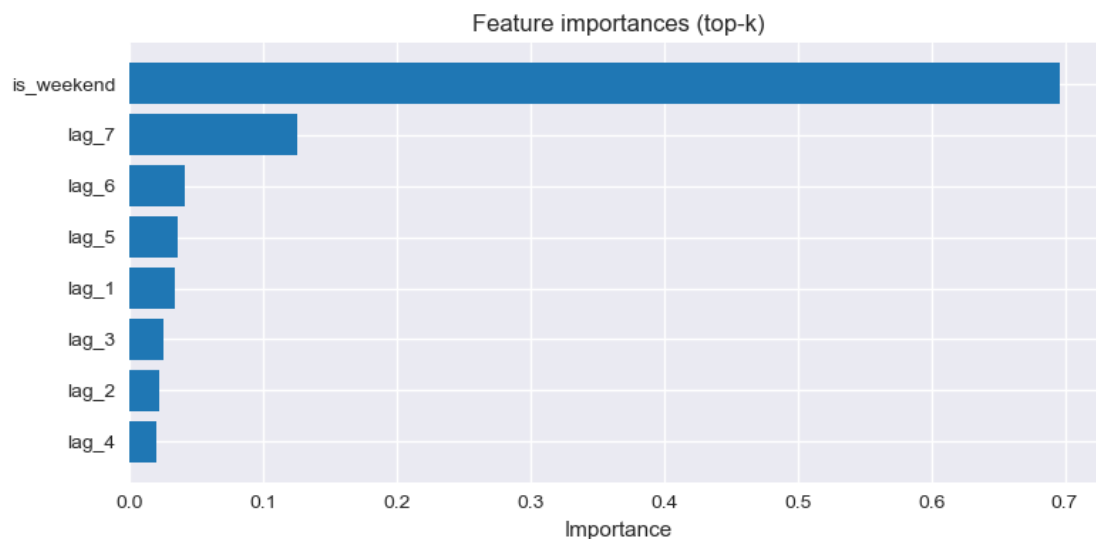
- Optionally aggregate lagged vs exogenous importance:

```
agg_imp = aggregate_importance(df_imp, lag_prefix='lag_')
display(agg_imp)
```

This helps to explain which past values and context variables drive forecasts or anomaly flags, and to connect ML behaviour to domain knowledge.

Example: feature importance on lagged demand and weekend indicator

We reuse the synthetic daily demand series with a weekend uplift, fit a RandomForest on lagged demand plus an `is_weekend` feature, and inspect feature importances.

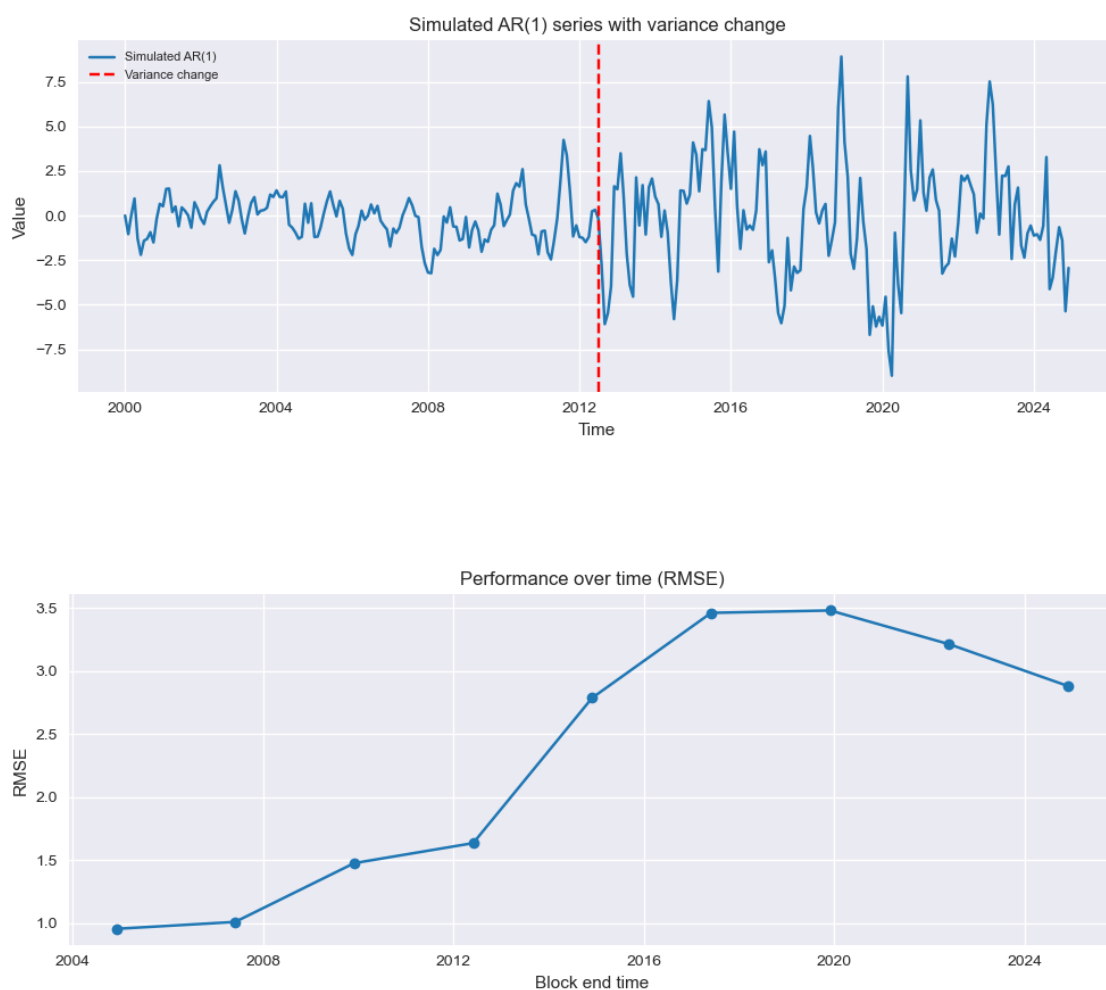


0.1.7 6. Simulation-based scaffolding for anomaly and drift studies

Simulation-based projects are explicitly acceptable for the time-series mini-project. This section provides a self-contained pattern for:

- simulating an AR(1) series with a change in variance (noise level);
- evaluating an ARIMA model block-wise over time;
- plotting both the simulated series and the performance curve.

You can adapt this pattern to construct more complex scenarios (for example level shifts, regime changes, or multivariate systems).



Usage pattern:

- Simulate a drifting or variance-change series:

```

series_sim = simulate_ar1_with_variance_change(
    n=300,
    phi=0.7,
    sigma1=1.0,
    sigma2=2.5,
    level=0.0,
    change_point=150,
    seed=42,
)

```

- Evaluate drift behaviour:

```
results_sim = block_forecast_evaluation_sim(series_sim, block_size=30, order=(1, 0, 0))
```

- Plot performance over time:

```
plot_block_performance(results_sim, metric='RMSE')
```

0.1.8 7. Walk-forward evaluation for ML models on lagged + exogenous features

For ML models on lagged+exogenous features (for example RandomForest or gradient boosting), we can use a simple **walk-forward evaluation** scheme:

- start with an initial training window;
- repeatedly refit the model on all available past data;
- evaluate on the next block of observations;
- move the window forward and repeat.

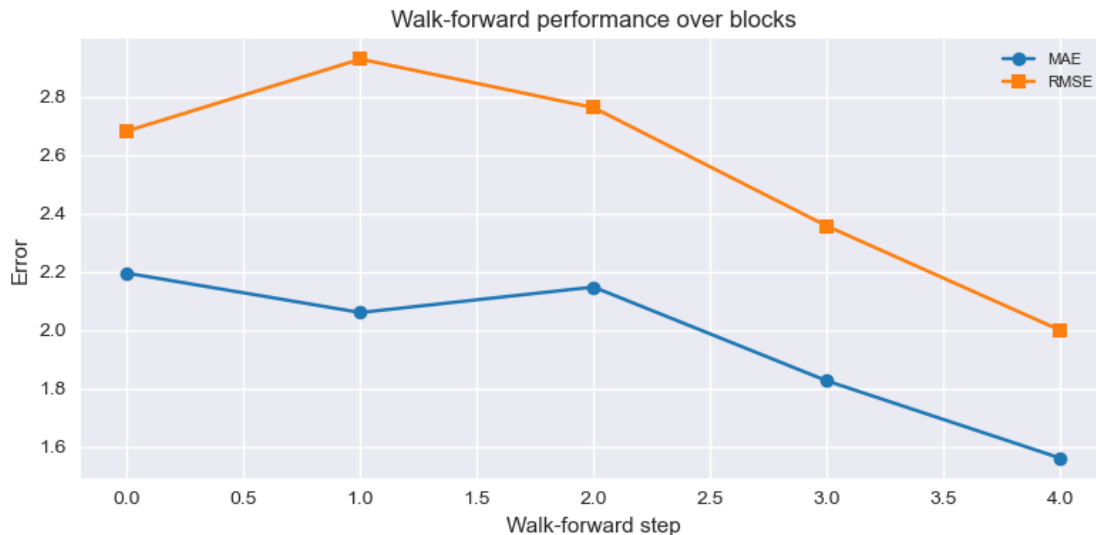
This mirrors model maintenance under concept drift: as more data arrive, we update the model and track performance over time. The function below is a generic walk-forward evaluator for any regressor with a scikit-learn style `.fit` / `.predict` interface.

Example: walk-forward evaluation of RandomForest on the weekend-demand data

We recreate the lagged+exogenous feature matrix (X, y) from the weekend-demand example in Section 4. We then:

- start with 200 days of training data,
- evaluate RandomForest on blocks of 30 days,
- move the boundary forward by 30 days each time,
- and plot how MAE and RMSE change over the walk-forward steps.

	start	end	MAE	RMSE
0	200	229	2.194628	2.681973
1	230	259	2.059637	2.928268
2	260	289	2.147112	2.761872
3	290	319	1.825535	2.356197
4	320	349	1.560831	1.999153



0.1.9 8. How to use Companion C in the time-series mini-project

For the time-series mini-project, you can use these templates in several ways:

- apply residual-based anomaly detection to a real or synthetic series;
- study concept drift by tracking performance over time on real or simulated data;
- explore multivariate dependence with cross-correlation and simple Granger-style tests;
- build ML models using lagged and exogenous/context features and examine feature importance.

Simulation-based investigations are explicitly acceptable. For example:

- generate synthetic series with known drift or anomaly patterns (for example level shifts, variance changes, or regime-specific dynamics);
- apply the anomaly and drift templates to study how different methods respond;
- construct multivariate synthetic data with known cross-dependencies and test whether cross-correlation and Granger-style tools recover the intended structure.

A strong mini-project does not need complicated models. It needs:

- clear problem framing and data description (Companions A and B, Lectures TS1–TS3);
- honest comparison against simple baselines;
- explicit discussion of drift, anomalies, and multivariate structure where relevant;
- sceptical evaluation of model behaviour, including cases where baselines are hard to beat.

Companion C provides reusable code for anomaly, drift, multivariate dependence, and ML-with-context workflows. You should adapt these templates to your own data or simulations and document what works, what does not, and why.